Evaluation Methodology Description:

Towards an Autonomous Lifecycle Agent for PDDL Domain Modelling

Student: Roshan Gupta | **Supervisor:** Prof. Dr. Ilche Georgievski

Institute of Architecture of Application Systems (IAAS), University of Stuttgart

This document defines the planned empirical evaluation methodology for the Master's Thesis *"Towards an Autonomous Lifecycle Agent for PDDL Domain Modelling"*. Addressing the requirements in the thesis description, it specifies the research questions and hypotheses, benchmark suite and fault generation protocols, baselines and ablation studies, quantitative metrics, and explicit alignment with the project objectives.

1. Research Questions and Hypotheses

Classical automated planning defines deterministic state-transition systems $\Sigma = (S, A, \gamma)$ within STRIPS and PDDL formalisms [1, 2]. While Large Language Models (LLMs) can parse natural language requirements, unconstrained generation frequently produces syntax errors, relational type mismatches, and reachability deadlocks. This evaluation investigates whether an autonomous agent orchestrating the domain modeling lifecycle—integrating symbolic oracles in an iterative closed loop—significantly improves model validity, repair efficiency, and regression resilience.

- **RQ1 (Representation & Syntax Decoupling):** Does intermediate Abstract Syntax Tree (AST) schema enforcement improve the first-pass syntactic validity and structural consistency of elicited PDDL domain models compared to unconstrained direct code generation?
  - *Hypothesis 1 (H_1):* Structurally constraining model synthesis to typed AST blueprints will yield a significantly higher initial Syntax Validity Rate (SVR) by preventing parenthesis mismatches, undeclared predicates, and invalid parameter typing prior to compilation.
- **RQ2 (Diagnostic Feedback vs. Naive Re-Prompting):** Does targeted diagnostic feedback derived from symbolic oracles resolve PDDL domain defects in fewer iterations and with higher overall success than re-prompting with raw compiler error logs?
  - *Hypothesis 2 (H_2):* Providing localized fault indications (isolating specific action schemas, missing preconditions, or ungrounded variables) will achieve higher Repair Pass@k rates ($k \leq 5$) and require fewer repair cycles than naive re-prompting with unparsed solver traces.
- **RQ3 (Continuous Evolution & Regression Prevention):** Does an automated regression testing mechanism prevent semantic breakages on historical problem instances during iterative domain evolution?
  - *Hypothesis 3 (H_3):* Gating domain evolution with a historical regression suite will retain a significantly higher Regression Preservation Rate (RPR) across prior problem instances compared to ungated evolutionary modifications.

2. Benchmark Suite: Domains, Tasks, and Fault Injection

The proposed benchmark suite (**PDDLLifeCycleBench**) is constructed from established classical planning domains from the International Planning Competitions (IPC) [2, 3]. It comprises **18 evaluation tasks** across **6 canonical domain families**: **(1) Blocksworld (IPC-2000):** Table, block stacking/unstacking, clear, and gripper arm relations; **(2) Logistics (IPC-2000):** Multi-city package transport across trucks, airplanes, and airport hubs; **(3) Gripper (IPC-1998):** Multi-room object transport constrained by two-arm robotic manipulator capacity; **(4) Depots (IPC-2002):** Coupled domain combining Blocksworld stacking constraints with Logistics freight transport; **(5) Rovers (IPC-2003):** Waypoint traversal, soil/rock sampling, camera calibration, and satellite communication windows; **(6) Satellite (IPC-2002):** Multi-spacecraft observation tasking with instrument calibration and power constraints.

2.1. Lifecycle Task Types and Systematic Fault Generation

Each domain family is evaluated across three core operational activities reflecting the engineering lifecycle:

(a) **Natural Language Elicitation:** Generating complete PDDL domain and problem specifications from natural language operational requirements, evaluated against reference IPC ground-truth models.

(b) **Systematic Fault Repair:** Evaluating the agent on domains injected with isolated, documented defects categorized according to recent empirical repair taxonomies [7]:
- *Precondition Omissions:* Dropping necessary guarding conditions (e.g., omitting `(at ?t ?l)` from `load-truck` in Logistics; omitting `(calibrated ?i)` from `take_image` in Satellite).
- *Corrupted/Inverted State Effects:* Incorrect state transitions (e.g., asserting `(not (at ?p ?l))` without asserting `(in ?p ?t)` in Logistics; omitting `(empty ?g)` upon drop in Gripper).
- *Type & Signature Inconsistencies:* Binding variables to incompatible object types or violating predicate arity.
- *Unbound Variables:* Introducing action effects that reference variables not declared in action parameters.

Each faulty benchmark instance is deterministically labeled with its ground-truth defect category, target action schema, and expected corrective action, enabling objective diagnostic accuracy measurement.

(c) **Continuous Domain Evolution:** Modifying existing valid domains to support new operational requirements (e.g., adding multi-package transport in Logistics, or dual-sensor inspection in Rovers) while verifying backward compatibility against historical task suites.

3. Baselines and Ablation Studies

To empirically isolate the contributions of the agent's core components, the system is evaluated against two baseline approaches and three targeted ablations under identical base LLM backends (e.g., Claude 3.5 Sonnet / GPT-4o, fixed temperature $T = 0.0$):

1. **Baseline 1 (Zero-Shot Direct Generation):** Unconstrained generation where the LLM is prompted with natural language requirements to emit raw PDDL code directly in a single pass without intermediate schemas or validation

loops.

2. **Baseline 2 (Naive Iterative Re-Prompting):** An iterative repair loop where raw compiler logs (from VAL or Fast Downward) are fed directly back into the LLM context without structured diagnostic localization [7].
3. **Ablation 1 (Without AST Schema / Direct PDDL Manipulation):** The agent maintains the iterative oracle feedback loop but edits raw PDDL text directly rather than operating over structured AST blueprints, isolating the value of the schema.
4. **Ablation 2 (Without Diagnostic Localization):** The agent receives only binary pass/fail verification outcomes without localized defect triage hints, isolating the impact of structured fault localization.
5. **Ablation 3 (Evolution Without Regression Gating):** The agent modifies domain models to accommodate new operational requirements without running historical regression checks, measuring susceptibility to backward-incompatible breakages.

4. Evaluation Metrics and Scientific Rationale

The following metrics will be measured across all benchmark tasks to evaluate the hypotheses:

Metric	Operational Definition	Scientific Rationale
Syntax Validity (SVR)	$\text{SVR} = \frac{\text{Valid Domains}}{\text{Total Domains}} \times 100\%$ via VAL	Evaluates compliance with formal PDDL 2.1 syntax, typing, and predicate declarations [3, 4].
Solvability Rate (SR)	$\text{SR} = \frac{\text{Solvable Problems}}{\text{Total Problems}} \times 100\%$ via Fast Downward	Measures semantic reachability: whether a heuristic search planner finds a valid plan within timeout ($t = 180\,\text{s}$) [5].
Plan Validity (PVR)	$\text{PVR} = \frac{\text{Valid Plans}}{\text{Synthesized Plans}} \times 100\%$ via VAL	Verifies that synthesized action trajectories execute without constraint violations under formal state transitions Σ [1, 4].
Repair Pass@k	Success rate within $k \in \{1, 2, 3, 5\}$ repair cycles	Quantifies convergence speed and repair effectiveness under symbolic oracle feedback.
Mean Iterations ($\bar{I}$)	Average repair cycles required to reach $\text{SR} = 1$	Measures convergence efficiency and operational token economy during model repair.
Structural F_1 Score	Precision, Recall, and F_1 of predicates, parameters, and action schemas vs. ground truth	Quantifies relational accuracy against reference IPC domain implementations [7].
Regression Preservation (RPR)	$\text{RPR} = \frac{\text{Solvable Historical Cases (Post-Evol.)}}{\text{Total Historical Cases}} \times 100\%$	Evaluates whether domain modifications preserve backward compatibility on pre-existing problem instances.

5. Relation to the Official Thesis Description

The proposed evaluation methodology directly operationalizes the core objectives of the approved thesis description:

1. **Full Lifecycle Coverage:** It evaluates the key phases of the Knowledge Engineering for Planning and Scheduling (KEPS) Software Development Life Cycle [6]: Elicitation, Synthesis, Validation, Diagnostic Repair, and Evolution.
2. **Grounded Neurosymbolic Integration:** Rather than relying on unverified LLM self-reflection, it integrates standard classical tools (VAL validator [4] and Fast Downward planner [5]) as sound, external ground-truth oracles.
3. **Standardized Benchmarking:** Utilizing six established IPC domain families provides objective, reproducible empirical results that directly answer the main thesis research questions.

Verified References

[1] M. Ghallab, D. Nau, and P. Traverso. *Automated Planning: Theory & Practice*. Morgan Kaufmann, 2004. ISBN: 978-1-55860-856-5.

[2] D. McDermott et al. PDDL—The Planning Domain Definition Language. Technical Report CVC TR-98-003, Yale University, 1998.

[3] M. Fox and D. Long. PDDL2.1: An extension to PDDL for expressing temporal and numeric planning domains. *Journal of Artificial Intelligence Research*, 20:61–124, 2003. doi:10.1613/jair.1129.

[4] R. Howey, D. Long, and M. Fox. VAL: Automatic plan validation, continuous effects and mixed initiative planning using PDDL. In *Proc. 16th IEEE Intl. Conf. on Tools with Artificial Intelligence (ICTAI)*, pp. 294–301, 2004.

[5] M. Helmert. The Fast Downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006. doi:10.1613/jair.1705.

[6] I. Georgievski. Software development lifecycle for engineering AI planning systems. In *Proc. 18th Intl. Conf. on Software Technologies (ICSOFT)*, pp. 751–760, 2023. doi:10.5220/0012149100003538.

[7] N. K. Bavandpour and P. Bercher. LLM-Only PDDL domain repair with open-weight models. *arXiv preprint arXiv:2603.04221*, 2026.